SortingComparison.java

```java
import java.util.Random;

public class SortingComparison {

    static final int SIZE = 1000;
    static final int RUNS = 5;

    // Generate random array
    static int[] generateArray() {
        Random random = new Random();
        int[] arr = new int[SIZE];

        for (int i = 0; i < SIZE; i++) {
            arr[i] = random.nextInt(10000);
        }

        return arr;
    }

    // Copy array
    static int[] copyArray(int[] arr) {
        int[] copy = new int[arr.length];

        for (int i = 0; i < arr.length; i++) {
            copy[i] = arr[i];
        }

        return copy;
    }

    // Selection Sort
    static void selectionSort(int[] arr) {
        for (int i = 0; i < arr.length - 1; i++) {
            int min = i;

            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[min]) {
                    min = j;
                }
            }

            int temp = arr[i];
            arr[i] = arr[min];
            arr[min] = temp;
        }
    }

    // Insertion Sort
    static void insertionSort(int[] arr) {
        for (int i = 1; i < arr.length; i++) {
            int key = arr[i];
            int j = i - 1;

            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }

            arr[j + 1] = key;
        }
    }

    // Bubble Sort
    static void bubbleSort(int[] arr) {
```

```
        for (int i = 0; i < arr.length - 1; i++) {
            boolean swapped = false;

            for (int j = 0; j < arr.length - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swapped = true;
                }
            }

            if (!swapped) {
                break;
            }
        }
    }

    // Merge Sort
    static void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;

            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);

            merge(arr, left, mid, right);
        }
    }

    static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] L = new int[n1];
        int[] R = new int[n2];

        for (int i = 0; i < n1; i++) {
            L[i] = arr[left + i];
        }

        for (int j = 0; j < n2; j++) {
            R[j] = arr[mid + 1 + j];
        }

        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) {
                arr[k++] = L[i++];
            } else {
                arr[k++] = R[j++];
            }
        }

        while (i < n1) {
            arr[k++] = L[i++];
        }

        while (j < n2) {
            arr[k++] = R[j++];
        }
    }

```

```
    // Check whether array is sorted
    static boolean isSorted(int[] arr) {
        for (int i = 1; i < arr.length; i++) {
            if (arr[i - 1] > arr[i]) {
                return false;
            }
        }
        return true;
    }

    // Measure average execution time
    static double measureTime(int algorithm, int[] original) {
        long totalTime = 0;

        for (int r = 0; r < RUNS; r++) {
            int[] arr = copyArray(original);

            long start = System.nanoTime();

            switch (algorithm) {
                case 0:
                    selectionSort(arr);
                    break;
                case 1:
                    insertionSort(arr);
                    break;
                case 2:
                    bubbleSort(arr);
                    break;
                case 3:
                    mergeSort(arr, 0, arr.length - 1);
                    break;
            }

            long end = System.nanoTime();

            if (!isSorted(arr)) {
                throw new RuntimeException("Sorting failed!");
            }

            totalTime += end - start;
        }

        return totalTime / (double) RUNS / 1_000_000.0;
    }

    public static void main(String[] args) {
        int[] original = generateArray();

        String[] names = {
            "Selection Sort",
            "Insertion Sort",
            "Bubble Sort",
            "Merge Sort"
        };

        double[] times = new double[4];

        System.out.println("Array Size: " + SIZE);
        System.out.println("Number of Runs: " + RUNS);
        System.out.println();

        System.out.printf("%-20s %15s%n",
                "Algorithm", "Average Time (ms)");
```

SortingComparison.java

```
193
194         System.out.println("---------------------------------------");
195
196         for (int i = 0; i < 4; i++) {
197             times[i] = measureTime(i, original);
198
199             System.out.printf("%-20s %15.4f%n",
200                     names[i], times[i]);
201         }
202
203         System.out.println("---------------------------------------");
204
205         int fastest = 0;
206
207         for (int i = 1; i < 4; i++) {
208             if (times[i] < times[fastest]) {
209                 fastest = i;
210             }
211         }
212
213         System.out.println("Fastest Algorithm: " + names[fastest]);
214         System.out.println("All arrays sorted successfully.");
215     }
216 }
217
```